

```
"""
```

```
physics.py - road signal interpolation, asymmetric damper, EOM, static equilibr
```

```
Matches your working base_final.py exactly:
```

- load\_track uses skiprows=2
- geom\_constraints uses s1/s2 (cfg["s1"], cfg["s2"])
- build\_M\_R computes k\_tf/K\_r1/K\_r2 internally from K\_f/K\_2/K\_3 and S1/S2/S3
- front damper scaled by C\_f:  $F_{df} = C_f * \text{damper.force}(v_f)$
- static\_equilibrium matches static\_equilibrium\_state exactly

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from typing import Callable, Dict, Optional, Tuple
```

```
import numpy as np
```

```
import pandas as pd
```

```
from scipy.integrate import solve_ivp
```

```
from scipy.linalg import solve as lin_solve
```

```
from scipy.optimize import least_squares
```

```
from config import BASE_CFG, DT, STATE_NAMES, ZC, THC, PHC, ZS, THS, PHS
```

```
NDOF = 6
```

```
NC = 2
```

```
# -----  
# Asymmetric two-stage damper (matches your dataclass exactly)  
# -----
```

```
@dataclass
```

```
class TwoStageAsymmetricDamper:
```

```
    cs_minus: float
```

```
    asym_ratio: float
```

```
    gamma_c: float
```

```
    gamma_r: float
```

```
    alpha_c: float = -0.05
```

```
    alpha_r: float = 0.13
```

```
    def force(self, v_rel: float) -> float:
```

```
        c_plus = self.asym_ratio * self.cs_minus
```

```
        if v_rel < 0.0:
```

```
            if v_rel >= self.alpha_c:
```

```

        return self.cs_minus * v_rel
    else:
        return self.cs_minus * (
            self.alpha_c + self.gamma_c * (v_rel - self.alpha_c)
        )
    else:
        if v_rel <= self.alpha_r:
            return c_plus * v_rel
        else:
            return c_plus * (
                self.alpha_r + self.gamma_r * (v_rel - self.alpha_r)
            )

# -----
# Road signals (skiprows=2 matches your CSV format)
# -----

def load_track(csv_path: str) -> Tuple[np.ndarray, np.ndarray]:
    """Read 2-column CSV with 2 header rows to skip."""
    df = pd.read_csv(csv_path, skiprows=2, header=None)
    t = pd.to_numeric(df.iloc[:, 0], errors="coerce").values
    z = pd.to_numeric(df.iloc[:, 1], errors="coerce").values
    mask = np.isfinite(t) & np.isfinite(z)
    return t[mask].astype(float), z[mask].astype(float)

def make_linear_interp(x: np.ndarray, y: np.ndarray) -> Callable:
    x, y = np.asarray(x), np.asarray(y)
    def f(xq):
        xq = np.asarray(xq)
        xq_c = np.clip(xq, x[0], x[-1])
        idx = np.clip(np.searchsorted(x, xq_c) - 1, 0, len(x) - 2)
        x0, x1 = x[idx], x[idx + 1]
        y0, y1 = y[idx], y[idx + 1]
        w = (xq_c - x0) / np.maximum(x1 - x0, 1e-12)
        return y0 * (1 - w) + y1 * w
    return f

@dataclass
class RoadSignals:
    f1L: Callable; f1R: Callable
    f2L: Callable; f2R: Callable
    f3L: Callable; f3R: Callable

    def axle_inputs(self, t: float, cfg: Dict) -> Tuple[float, ...]:

```

```

        zr1L, zr1R = float(self.f1L(t)), float(self.f1R(t))
        zr2L, zr2R = float(self.f2L(t)), float(self.f2R(t))
        zr3L, zr3R = float(self.f3L(t)), float(self.f3R(t))
        z1f = 0.5 * (zr1L + zr1R)
        z2 = 0.5 * (zr2L + zr2R)
        z3 = 0.5 * (zr3L + zr3R)
        ph_f = (zr1L - zr1R) / cfg["WT1"]
        ph2 = (zr2L - zr2R) / cfg["WT2"]
        ph3 = (zr3L - zr3R) / cfg["WT3"]
        return float(z1f), float(ph_f), float(z2), float(ph2), float(z3), float(p

def axle_input_rates(self, t: float, cfg: Dict,
                    dt: float = DT) -> Tuple[float, ...]:
    if dt is None:
        dt = DT
    p = self.axle_inputs(t + dt, cfg)
    m = self.axle_inputs(t - dt, cfg)
    return tuple((a - b) / (2.0 * dt) for a, b in zip(p, m))

def build_road_signals(cfg: Dict) -> RoadSignals:
    t1L, z1L = load_track(cfg["axlefront_left_csv"])
    t1R, z1R = load_track(cfg["axlefront_right_csv"])
    t2L, z2L = load_track(cfg["axlerear1_left_csv"])
    t2R, z2R = load_track(cfg["axlerear1_right_csv"])
    t3L, z3L = load_track(cfg["axlerear2_left_csv"])
    t3R, z3R = load_track(cfg["axlerear2_right_csv"])
    return RoadSignals(
        make_linear_interp(t1L, z1L), make_linear_interp(t1R, z1R),
        make_linear_interp(t2L, z2L), make_linear_interp(t2R, z2R),
        make_linear_interp(t3L, z3L), make_linear_interp(t3R, z3R),
    )

def precompute_road_array(cfg: Dict, t_eval: np.ndarray) -> np.ndarray:
    """Pre-evaluate all 6 axle signals. Returns float32 [T, 6]."""
    road = build_road_signals(cfg)
    out = np.empty((len(t_eval), 6), dtype=np.float32)
    for i, t in enumerate(t_eval):
        out[i] = road.axle_inputs(t, cfg)
    return out

# -----
# Geometric constraints (uses s1/s2 from cfg, matching your code)
# -----

```

```

def geom_constraints(
    q: np.ndarray,    # [6]
    t: float,
    cfg: Dict,
    road: RoadSignals,
) -> Tuple[np.ndarray, np.ndarray]:
    z_s = q[ZS]; th_s = q[THS]; ph_s = q[PHS]
    _, _, z2, ph2, z3, ph3 = road.axle_inputs(t, cfg)

    l2 = cfg["L12"]
    l3 = cfg["L12"] + cfg["L23"]
    S2 = cfg["S_tf2"]; S3 = cfg["S_tf3"]
    sl2 = cfg["s1"]      # torque rod length axle 2 (your code uses s1/s2)
    sl3 = cfg["s2"]      # torque rod length axle 3
    bL2 = cfg["beta_L2"]; bL3 = cfg["beta_L3"]

    g2 = (z_s + l2*th_s + S2*ph_s
          - sl2 * np.sin(bL2 - th_s)
          - (z2 + 0.5*cfg["WT2"]*ph2))
    g3 = (z_s + l3*th_s + S3*ph_s
          - sl3 * np.sin(bL3 - th_s)
          - (z3 + 0.5*cfg["WT3"]*ph3))

    G = np.zeros((NC, NDOF))
    G[0, ZS] = 1.0
    G[0, THS] = l2 + sl2 * np.cos(bL2 - th_s)
    G[0, PHS] = S2
    G[1, ZS] = 1.0
    G[1, THS] = l3 + sl3 * np.cos(bL3 - th_s)
    G[1, PHS] = S3

    return np.array([g2, g3], dtype=float), G

```

---

```

#
# Mass matrix [6,6] and generalised force vector [6]
# Matches your build_M_R exactly, including:
#   - F_df = C_f * damper.force(v_f)    (C_f scales the damper)
#   - k_tf/K_r1/K_r2 computed from K_f/K_2/K_3 and S1/S2/S3
#

```

---

```

def build_M_R(
    q: np.ndarray,    # [6]
    v: np.ndarray,    # [6]
    t: float,
    cfg: Dict,
    road: RoadSignals,

```

```

) -> Tuple[np.ndarray, np.ndarray]:

z_c = q[ZC]; th_c = q[THC]; ph_c = q[PHC]
z_s = q[ZS]; th_s = q[THS]; ph_s = q[PHS]
dz_c = v[ZC]; dth_c= v[THC]; dph_c= v[PHC]
dz_s = v[ZS]; dth_s= v[THS]; dph_s= v[PHS]

z1f, ph_f, z2, ph2, z3, ph3 = road.axle_inputs(t, cfg)
dz1f, dph_f, dz2, dph2, dz3, dph3 = road.axle_input_rates(t, cfg)

phi_NRS2 = ((cfg["beta_L2"]*cfg["L_DL2"] - cfg["beta_R2"]*cfg["L_DR2"])/
             / max(cfg["S_tf2"], 1e-6))
phi_NRS3 = ((cfg["beta_L3"]*cfg["L_DL3"] - cfg["beta_R3"]*cfg["L_DR3"])/
             / max(cfg["S_tf3"], 1e-6))

m_c = cfg["m_c"]; I_xxc = cfg["I_xxc"]; I_yyc = cfg["I_yyc"]
m_s = cfg["m_s"]; I_sxx = cfg["I_sxx"]
I_syy = cfg["I_syy"]; I_sxy = cfg["I_sxy"]
S1 = cfg["S_f"]; S2 = cfg["S_tf2"]; S3 = cfg["S_tf3"]
a, b = cfg["a"], cfg["b"]
hs, g = cfg["hs"], cfg["g"]
l_cfcg = cfg["l_cfcg"]; l_crcg = cfg["l_crcg"]
l_cf = cfg["l_cf"]; l_cr = cfg["l_cr"]
lf = cfg["lf"]; hcp = cfg["hcp"]
l2 = cfg["L12"]; l3 = cfg["L12"] + cfg["L23"]
beta_L2, beta_R2 = cfg["beta_L2"], cfg["beta_R2"]
beta_L3, beta_R3 = cfg["beta_L3"], cfg["beta_R3"]
L_DL2, L_DR2 = cfg["L_DL2"], cfg["L_DR2"]
L_DL3, L_DR3 = cfg["L_DL3"], cfg["L_DR3"]
Kcfl, Kcfr, Kcrl, Kcrr = cfg["K_cfl"], cfg["K_cfr"], cfg["K_crl"], cfg["K_crr"]
Ccfl, Ccfr, Ccrl, Ccrr = cfg["C_cfl"], cfg["C_cfr"], cfg["C_crl"], cfg["C_crr"]
K_f, C_f = cfg["K_f"], cfg["C_f"]
K_2, C_2 = cfg["K_2"], cfg["C_2"]
K_3, C_3 = cfg["K_3"], cfg["C_3"]

# — 6x6 mass matrix —————
M = np.zeros((NDOF, NDOF))
M[ZC, ZC] = m_c
M[THC, THC] = I_yyc
M[PHC, PHC] = I_xxc
M[ZS, ZS] = m_s
M[THS, THS] = I_syy
M[THS, PHS] = I_sxy
M[PHS, THS] = I_sxy
M[PHS, PHS] = I_sxx + m_s * hs**2

# — front asymmetric damper (C_f scales the damper, matching your code)

```

```

damper = TwoStageAsymmetricDamper(
    cs_minus=cfg["cs_minus"],
    asym_ratio=cfg["asym_ratio"],
    gamma_c=cfg["gamma_c"],
    gamma_r=cfg["gamma_r"],
)

v_f = dz_s - lf*dth_s - dzlf
F_df = C_f * damper.force(v_f)          # NOTE: C_f multiplies the normalised f

Csum = Ccfl + Ccfr + Ccrl + Ccrr
Ksum = Kcfl + Kcfr + Kcrl + Kcrr
R = np.zeros(NDOF)

R[ZC] = (
    + Csum*(dz_c - dz_s) + Ksum*(z_c - z_s)
    - (Ccfl*1_cfcg + Ccfr*1_cfcg - Ccrl*1_crcg - Ccrr*1_crcg)*dth_c
    - (-Ccfl*1_cf - Ccfr*1_cf - Ccrl*1_cr - Ccrr*1_cr)*dth_s
    - (-Ccfl*b + Ccfr*a - Ccrl*b + Ccrr*a)*dph_c
    - (Ccfl*b - Ccfr*a + Ccrl*b - Ccrr*a)*dph_s
    - (Kcfl*1_cfcg + Kcfr*1_cfcg - Kcrl*1_crcg - Kcrr*1_crcg)*th_c
    - (-Kcfl*1_cf - Kcfr*1_cf - Kcrl*1_cr - Kcrr*1_cr)*th_s
    - (-Kcfl*b + Kcfr*a - Kcrl*b + Kcrr*a)*ph_c
    - (Kcfl*b - Kcfr*a + Kcrl*b - Kcrr*a)*ph_s
)

R[THC] = (
    - (Ccfl*1_cfcg + Ccfr*1_cfcg - Ccrl*1_crcg - Ccrr*1_crcg)*dz_c
    - (-Ccfl*1_cfcg - Ccfr*1_cfcg - Ccrl*1_crcg - Ccrr*1_crcg)*dz_s
    - (Kcfl*1_cfcg + Kcfr*1_cfcg - Kcrl*1_crcg - Kcrr*1_crcg)*z_c
    - (-Kcfl*1_cfcg - Kcfr*1_cfcg - Kcrl*1_crcg - Kcrr*1_crcg)*z_s
    - (-Ccfl*1_cfcg**2 - Ccfr*1_cfcg**2 - Ccrl*1_crcg**2 - Ccrr*1_crcg**2)*dt
    - (Ccfl*1_cfcg*1_cf + Ccfr*1_cfcg*1_cf
        - Ccrl*1_crcg*1_cr - Ccrr*1_crcg*1_cr)*dth_s
    - (-Ccfl*1_cfcg*b + Ccfr*1_cfcg*a
        - Ccrl*1_crcg*b + Ccrr*1_crcg*a)*dph_c
    - (Ccfl*1_cfcg*b - Ccfr*1_cfcg*a
        + Ccrl*1_crcg*b - Ccrr*1_crcg*a)*dph_s
    - (-Kcfl*1_cfcg**2 - Kcfr*1_cfcg**2
        - Kcrl*1_crcg**2 - Kcrr*1_crcg**2 + m_c*g*hcp)*th_c
    - (Kcfl*1_cfcg*1_cf + Kcfr*1_cfcg*1_cf
        - Kcrl*1_crcg*1_cr - Kcrr*1_crcg*1_cr)*th_s
    - (-Kcfl*1_cfcg*b + Kcfr*1_cfcg*a
        - Kcrl*1_crcg*b + Kcrr*1_crcg*a)*ph_c
    - (Kcfl*1_cfcg*b - Kcfr*1_cfcg*a
        + Kcrl*1_crcg*b - Kcrr*1_crcg*a)*ph_s
)

```

```

R[PHC] = (
  - (-Ccfl*b + Ccfr*a - Ccrl*b + Ccrr*a)*dz_c
  - ( Ccfl*b - Ccfr*a + Ccrl*b - Ccrr*a)*dz_s
  - (-Kcfl*b + Kcfr*a - Kcrl*b + Kcrr*a)*z_c
  - ( Kcfl*b - Kcfr*a + Kcrl*b - Kcrr*a)*z_s
  - (-Ccfl*1_cfcg*b - Ccfr*1_cfcg*a
    + Ccrl*1_crcg*b + Ccrr*1_crcg*a)*dth_c
  - ( Ccfl*1_cfcg*b + Ccfr*1_cfcg*a
    - Ccrl*1_crcg*b - Ccrr*1_crcg*a)*dth_s
  - (-Ccfl*b**2 + Ccfr*a**2 - Ccrl*b**2 + Ccrr*a**2)*dph_c
  - ( Ccfl*b**2 - Ccfr*a**2 + Ccrl*b**2 - Ccrr*a**2)*dph_s
  - (-Kcfl*1_cfcg*b - Kcfr*1_cfcg*a
    + Kcrl*1_crcg*b + Kcrr*1_crcg*a)*th_c
  - ( Kcfl*1_cfcg*b + Kcfr*1_cfcg*a
    - Kcrl*1_crcg*b - Kcrr*1_crcg*a)*th_s
  - (-Kcfl*b**2 + Kcfr*a**2 - Kcrl*b**2 + Kcrr*a**2)*ph_c
  - ( Kcfl*b**2 - Kcfr*a**2 + Kcrl*b**2 - Kcrr*a**2)*ph_s
)

```

```

R[ZS] = (
  - (Ccfl + Ccfr + Ccrl + Ccrr)*dz_c
  - (-Ccfl*1_cfcg - Ccfr*1_cfcg + Ccrl*1_crcg + Ccrr*1_crcg)*dth_c
  - (-Ccfl - Ccfr - Ccrl - Ccrr)*dz_s
  - (Ccfl*1_cf + Ccfr*1_cf + Ccrl*1_cr + Ccrr*1_cr)*dth_s
  - (Kcfl + Kcfr + Kcrl + Kcrr)*z_c
  - (-Kcfl*1_cfcg - Kcfr*1_cfcg + Kcrl*1_crcg + Kcrr*1_crcg)*th_c
  - (-Kcfl - Kcfr - Kcrl - Kcrr)*z_s
  - (Kcfl*1_cf + Kcfr*1_cf + Kcrl*1_cr + Kcrr*1_cr)*th_s
  + K_f*(z_s - lf*th_s - zlf) + F_df
  + K_2*(z_s - z2 - beta_L2*L_DL2 - beta_R2*L_DR2 + l2*th_s)
  + C_2*(dz_s - dz2 + l2*dth_s)
  + K_3*(z_s - z3 - beta_L3*L_DL3 - beta_R3*L_DR3 + l3*th_s)
  + C_3*(dz_s - dz3 + l3*dth_s)
)

```

```

R[THS] = (
  - (Ccfl*1_cfcg + Ccfr*1_cfcg - Ccrl*1_crcg - Ccrr*1_crcg)*dz_c
  - (-Ccfl*1_cfcg**2 - Ccfr*1_cfcg**2
    - Ccrl*1_crcg**2 - Ccrr*1_crcg**2)*dth_c
  - (-Ccfl*1_cf - Ccfr*1_cf - Ccrl*1_cr - Ccrr*1_cr)*dz_s
  - (Ccfl*1_cfcg*1_cf + Ccfr*1_cfcg*1_cf
    - Ccrl*1_crcg*1_cr - Ccrr*1_crcg*1_cr)*dth_s
  - (Kcfl*1_cf + Kcfr*1_cf + Kcrl*1_cr + Kcrr*1_cr)*z_c
  - (-Kcfl*1_cfcg*1_cf - Kcfr*1_cfcg*1_cf
    + Kcrl*1_crcg*1_cr + Kcrr*1_crcg*1_cr)*th_c
  - (-Kcfl*1_cf - Kcfr*1_cf - Kcrl*1_cr - Kcrr*1_cr)*z_s
  - (Kcfl*1_cf**2 + Kcfr*1_cf**2

```

```

        + Kcrl*l_cr**2 + Kcrr*l_cr**2)*th_s
    - lf*(K_f*(z_s - lf*th_s - z1f) + F_df)
    + l2*(K_2*(z_s - z2 - beta_L2*L_DL2 - beta_R2*L_DR2 + l2*th_s)
        + C_2*(dz_s - dz2 + l2*dth_s))
    + l3*(K_3*(z_s - z3 - beta_L3*L_DL3 - beta_R3*L_DR3 + l3*th_s)
        + C_3*(dz_s - dz3 + l3*dth_s))
)

# Roll stiffness/damping computed from primary suspension (matches your code)
k_tf = 0.5 * K_f * S1**2
K_r1 = 0.5 * K_2 * S2**2
K_r2 = 0.5 * K_3 * S3**2
C_tf = 0.5 * C_f * S1**2
C_r1 = 0.5 * C_2 * S2**2
C_r2 = 0.5 * C_3 * S3**2

R[PHS] = -(
    + m_s*g*hs*ph_s
    - k_tf*(ph_s - ph_f) - C_tf*(dph_s - dph_f)
    - K_r1*(ph_s - ph2 - phi_NRS2) - C_r1*(dph_s - dph2)
    - K_r2*(ph_s - ph3 - phi_NRS3) - C_r2*(dph_s - dph3)
)

return M, R

# -----
# KKT solve 8x8 system
# -----

def kkt_solve(M, R, G, gamma):
    n, nc = NDOF, NC
    A = np.zeros((n + nc, n + nc))
    b = np.zeros(n + nc)
    A[:n, :n] = M; A[:n, n:] = G.T; A[n:, :n] = G
    b[:n] = -R; b[n:] = -gamma
    return lin_solve(A, b)

# -----
# ODE RHS (state length = 12)
# -----

def rhs(t: float, x: np.ndarray, cfg: Dict, road: RoadSignals) -> np.ndarray:
    q = x[:NDOF]; v = x[NDOF:]
    M, R = build_M_R(q, v, t, cfg, road)
    gq, G = geom_constraints(q, t, cfg, road)

```

```

w, zeta = cfg["baum_omega"], cfg["baum_zeta"]
gamma    = w**2 * gq + 2.0*zeta*w*(G @ v)
sol      = kkt_solve(M, R, G, gamma)
xdot     = np.zeros_like(x)
xdot[:NDOF] = v
xdot[NDOF:] = sol[:NDOF]
return xdot

# -----
# Static equilibrium (matches your static_equilibrium_state exactly)
# -----

def static_equilibrium(cfg: Dict, road: RoadSignals,
                       verbose: bool = True) -> np.ndarray:
    q_seed = np.zeros(NDOF)
    lam_seed = np.zeros(NC)
    y0 = np.hstack([q_seed, lam_seed]) # (8,)
    t0 = 0.0

    def F(y):
        q = y[:NDOF]; lam = y[NDOF:]
        v0 = np.zeros(NDOF)
        M, R = build_M_R(q, v0, t0, cfg, road)
        gq, G = geom_constraints(q, t0, cfg, road)
        return np.hstack([R + G.T @ lam, 1e3 * gq]) # (8,) alpha=1e3

    lsq = least_squares(F, y0, method="trf", loss="soft_l1",
                       xtol=1e-12, ftol=1e-12, gtol=1e-12, max_nfev=800)

    if lsq.success:
        q0 = lsq.x[:NDOF]
        if verbose:
            g0, G0 = geom_constraints(q0, t0, cfg, road)
            M0, R0 = build_M_R(q0, np.zeros(NDOF), t0, cfg, road)
            eq_dyn0 = R0 + G0.T @ lsq.x[NDOF:]
            print(f" Static eq. (LSQ) ||g||={np.linalg.norm(g0):.3e} "
                  f"||R+G^T*lam||={np.linalg.norm(eq_dyn0):.3e}")
        return np.hstack([q0, np.zeros(NDOF)]) # (12,)

    if verbose:
        print(f" Static eq. LSQ failed ('{lsq.message}'); trying dynamic relaxat

    cfg_r = dict(cfg)
    cfg_r["C_f"] = cfg["C_f"] # keep C_f as-is (matches your code)
    cfg_r["C_2"] = cfg["C_2"] * 20
    cfg_r["C_3"] = cfg["C_3"] * 20

```

```

    cfg_r["C_cfl"] = cfg["C_cfl"] * 20
    cfg_r["C_cfr"] = cfg["C_cfr"] * 20
    cfg_r["C_crl"] = cfg["C_crl"] * 20
    cfg_r["C_crr"] = cfg["C_crr"] * 20

    x_init = np.zeros(2 * NDOF)
    sol = solve_ivp(lambda t, x: rhs(t, x, cfg_r, road),
                    (0.0, 3.0), x_init, method="Radau",
                    rtol=1e-7, atol=1e-9)
    q_relax = sol.y[:NDOF, -1]

    y_pol = np.hstack([q_relax, np.zeros(NC)])
    lsq2 = least_squares(F, y_pol, method="trf", loss="soft_l1",
                        xtol=1e-12, ftol=1e-12, gtol=1e-12, max_nfev=400)
    q0 = lsq2.x[:NDOF] if lsq2.success else q_relax
    if verbose:
        gn = np.linalg.norm(geom_constraints(q0, t0, cfg, road)[0])
        print(f" Static eq. (relax) ||g||={gn:.3e}")
    return np.hstack([q0, np.zeros(NDOF)]) # (12,)

# -----
# Single ODE case
# -----

def run_one_case(
    params: Dict,
    cfg_base: Dict,
    t_eval: np.ndarray,
    verbose: bool = False,
) -> Optional[pd.DataFrame]:
    cfg = {**cfg_base, **params}
    road = build_road_signals(cfg)

    if verbose:
        print(" Static equilibrium ...")
    try:
        x0 = static_equilibrium(cfg, road, verbose=verbose)
    except Exception as e:
        print(f" Static eq. failed: {e}")
        return None

    try:
        sol = solve_ivp(
            fun=lambda t, x: rhs(t, x, cfg, road),
            t_span=(t_eval[0], t_eval[-1]),
            y0=x0,

```

```

        t_eval=t_eval,
        method="Radau",
        max_step=0.01,
        rtol=1e-6,
        atol=1e-8,
    )
except Exception as e:
    print(f"    solve_ivp failed: {e}")
    return None

if sol.status != 0 or not np.all(np.isfinite(sol.y)):
    print(f"    solve_ivp: {sol.message}")
    return None

rows = []
for k in range(len(sol.t)):
    qk = sol.y[:NDOF, k]
    qdk = sol.y[NDOF:, k]
    tt = float(sol.t[k])
    M, R = build_M_R(qk, qdk, tt, cfg, road)
    gq, G = geom_constraints(qk, tt, cfg, road)
    w, zeta = cfg["baum_omega"], cfg["baum_zeta"]
    gamma = w**2*gq + 2.0*zeta*w*(G @ qdk)
    qddk = kkt_solve(M, R, G, gamma)[:NDOF]
    row = {"t": tt}
    row.update(params)
    for i, n in enumerate(STATE_NAMES):
        row[n] = float(qk[i])
        row[f"qd_{n}"] = float(qdk[i])
        row[f"qdd_{n}"] = float(qddk[i])
    rows.append(row)

return pd.DataFrame(rows)

# -----
# Seat RMS helper
# -----

def compute_seat_rms(df: pd.DataFrame, cfg: Dict,
                    t_ignore: float = 0.5) -> float:
    """
    ISO 2631-style combined 3-axis seat-point acceleration RMS.

    z-axis (vertical):      az = qdd_z_c
    x-axis (longitudinal): ax = -hcp * qdd_th_c    (pitch at seat height)
    y-axis (lateral):      ay = hcp * qdd_ph_c    (roll at seat height)

```

```
RMS_total = sqrt( mean(az2) + mean(ax2) + mean(ay2) )
```

Uses hcp (cabin pitch centre height) as the effective seat lever arm.  
"""

```
from config import T_IGNORE
t_ign = max(t_ignore, T_IGNORE)
mask = df["t"].values >= t_ign
if not mask.any():
    return float("nan")
h = cfg.get("hcp", 0.1)
az = df["qdd_z_c"].values[mask]
ax = -h * df["qdd_th_c"].values[mask]
ay = h * df["qdd_ph_c"].values[mask]
return float(np.sqrt(np.mean(az**2) + np.mean(ax**2) + np.mean(ay**2)))
```